

WareFlow

Warehouse Management System

Project Report

Submitted by: SHUBHANGI RAO
Registration Number: 25BAI10723

1. Introduction

WareFlow is a full-stack Warehouse Management System designed to support the complete movement of inventory through a warehouse operation. It connects product cataloguing, supplier management, warehouse capacity tracking, stock receiving, stock dispatch, purchase orders, alerts, dashboard analytics, and reporting through a browser-based interface.

The solution uses a decoupled architecture: a React and TypeScript frontend communicates with a Java Spring Boot REST API. The backend applies business rules and persists data through Spring Data JPA / Hibernate in a MySQL database. The supplied source also includes an in-app source-code viewer for browsing selected backend source categories.

Project objectives

- Centralize product, warehouse, supplier, and inventory movement information.
 - Reduce stock inaccuracies through validation during inward and outward transactions.
- Provide visibility into low stock, out-of-stock products, pending purchase orders, and warehouse utilization.
- Support reporting for inventory valuation, warehouse utilization, and movement history.
 - Offer a maintainable layered backend and a responsive modular frontend.

2. Problem Statement

Manual or fragmented warehouse operations can result in inaccurate stock counts, delayed receiving and dispatch records, poor visibility into storage capacity, and missed replenishment needs. When product, supplier, purchase-order, and movement records are maintained separately, users may not have a consistent view of the current inventory state.

WareFlow addresses this problem by maintaining related records in a central database and exposing operational workflows through a single user interface. The backend validates capacity and stock constraints before changing inventory, while the dashboard and alerts provide a summary of operational conditions.

Project boundaries

- The supplied implementation focuses on warehouse inventory and related procurement records.
- The database is relational and configured for MySQL.
- Authentication endpoints and roles are present, but the supplied security configuration currently permits API requests; production-grade authorization requires further hardening.
- The repository README states that automated tests are not currently included; manual and API-level verification are therefore important.

3. System Functional Requirements

Requirement ID	Module / Feature	Detailed Description
FR-01	Dashboard	Display inventory KPIs, stock totals, supplier count, warehouse utilization, and trends.
FR-02	Product Management	Create, read, update, delete, search, categorize, and adjust product stock; enforce unique SKU.
FR-03	Warehouse Management	Maintain facilities, locations, managers, capacities, occupied space, and available space.
FR-04	Supplier Management	Maintain supplier identity, company, contact details, supplied products, and status.
FR-05	Stock Inward	Receive goods, validate available warehouse capacity, update stock and occupancy, and record an inward audit entry.
FR-06	Stock Outward	Dispatch goods, validate available product stock, update stock and occupancy, and record an outward entry.
FR-07	Purchase Orders	Create orders and transition them through pending, approved, received, or cancelled states.
FR-08	Alerts	Generate out-of-stock, low-stock, overstock, and pending purchase-order notifications.
FR-09	Reports	Provide inventory, warehouse, inward, outward, supplier, and low-stock reports; support CSV export in the UI.
FR-10	Authentication	Provide login and current-user endpoints with role information.

FR-11	Error Handling	Return structured errors for missing resources, insufficient stock, capacity violations, and invalid input.
FR-12	Source Viewer	Allow users to inspect selected project source categories inside the application.

4. System Non-Functional Requirements

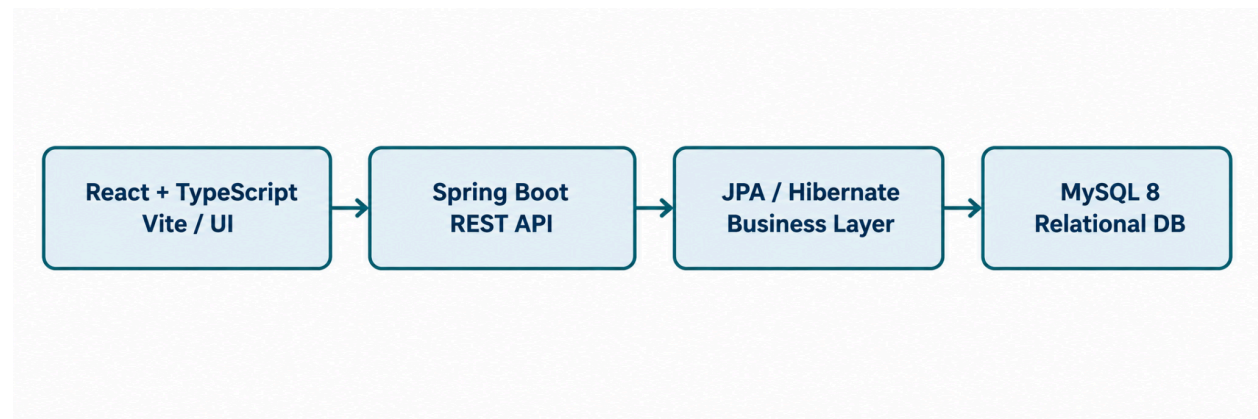
Category	Requirement / Design Implication
Performance	Use repository queries and indexed columns such as SKU, category, dates, and purchase-order status.
Reliability	Validate stock and capacity before committing inventory changes; use centralized exception handling.
Maintainability	Separate controllers, services, repositories, entities, DTOs, configuration, and frontend views.
Usability	Provide dashboard navigation, forms, status indicators, alerts, and toast notifications.
Scalability	Keep frontend and backend decoupled so the API can be consumed by other clients later.
Security	Use password encoding support and role concepts; strengthen authorization and token validation before production.
Data Integrity	Use relational foreign keys, unique SKU constraints, and explicit stock movement tables.
Portability	Run with Java 17+, Maven, Node.js 18+, and MySQL 8+ according to the project documentation.

Quality considerations

The project includes custom exception classes such as **InsufficientStockException**, **CapacityExceededException**, and **ResourceNotFoundException**. A global exception handler maps these failures to API responses. The frontend API wrapper also attempts to extract a message or error field from non-success responses and presents the result through toast notifications.

5. System Architecture

WareFlow follows a layered full-stack architecture. The frontend is responsible for navigation, forms, local UI state, and calling the REST API. The backend exposes resource-oriented endpoints, applies business logic in services, and persists entities through Spring Data JPA repositories. MySQL stores the operational records.



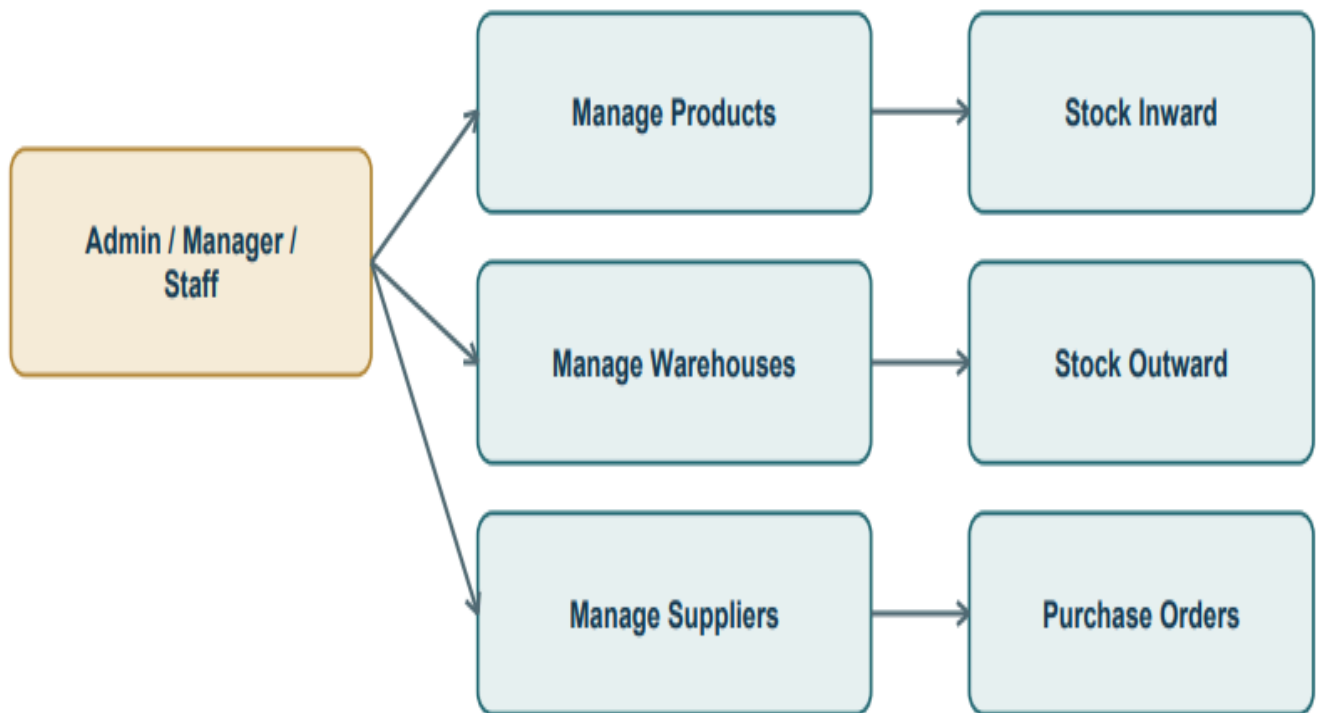
Backend layers

- **Controller layer:** REST endpoints for products, warehouses, suppliers, inward/outward movement, purchase orders, dashboard, reports, and authentication.

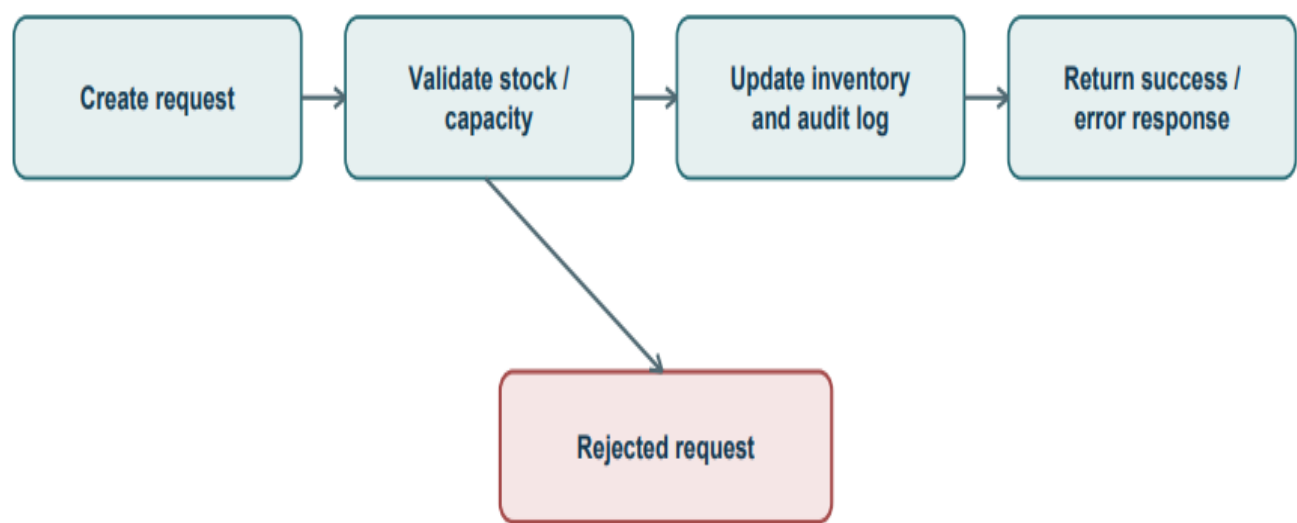
- **Service layer:** Business rules, validation, inventory updates, alert generation, and reporting.
- **Repository layer:** Spring Data JPA repositories for database access and custom query methods.
- **Entity layer:** Product, Warehouse, Supplier, StockInward, StockOutward, PurchaseOrder, and User models.
- **DTO layer:** Request and response structures used to avoid exposing every entity field directly.
- **Configuration / exception layer:** CORS, security configuration, and centralized error responses.

6. Design Diagrams

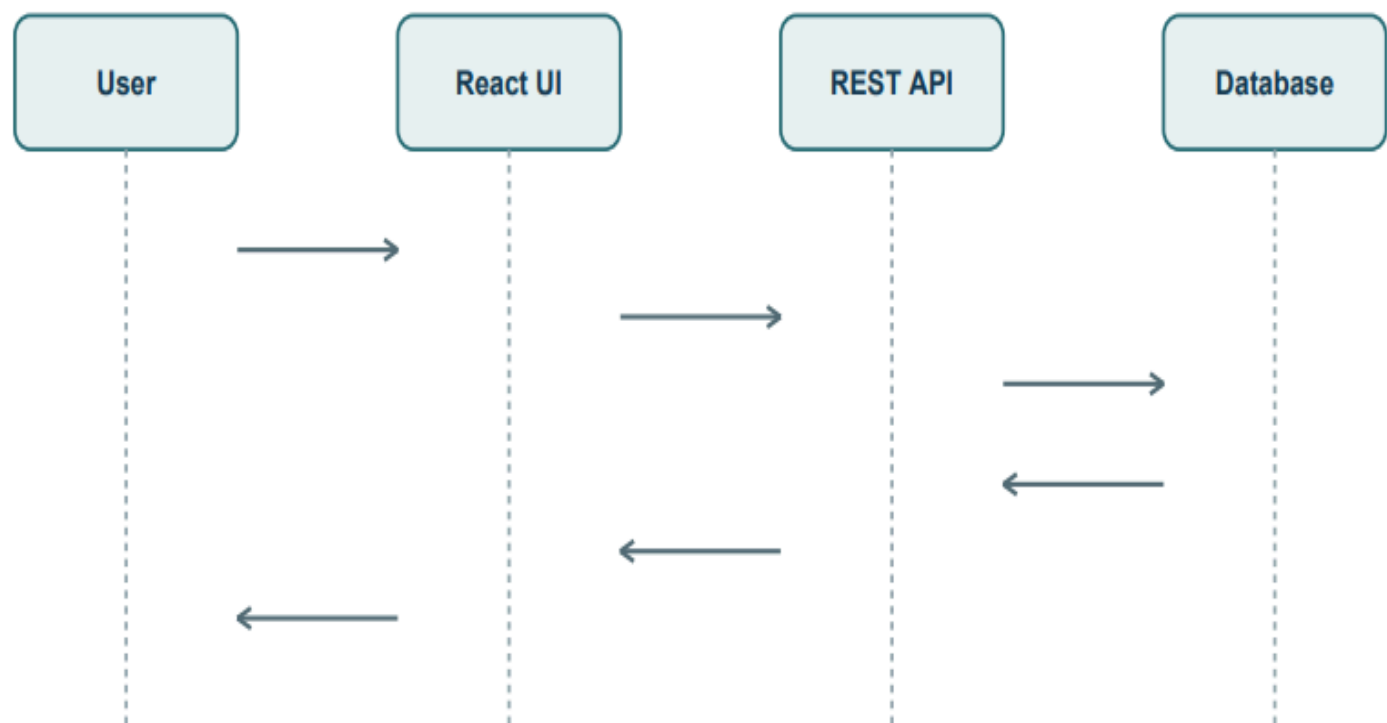
Case Diagram



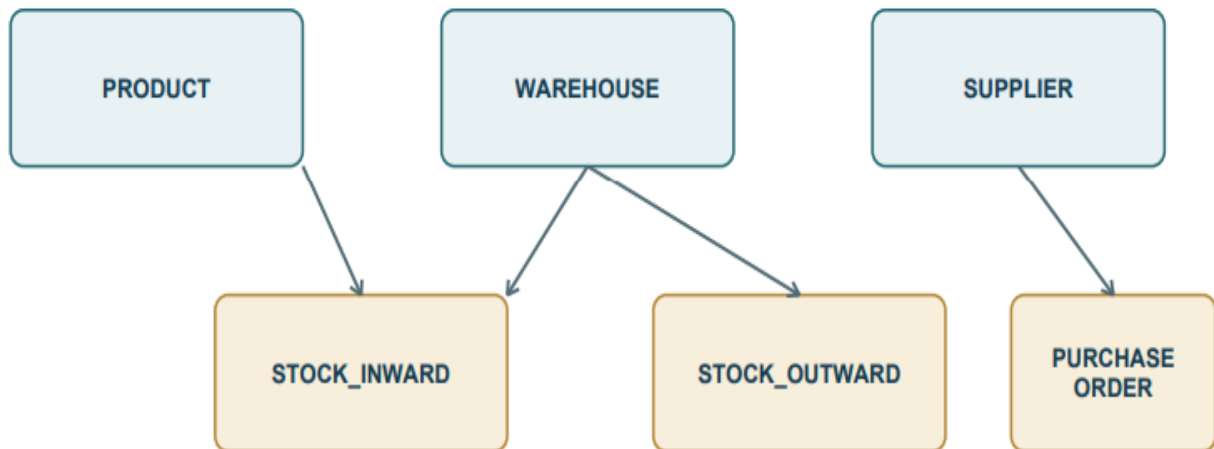
Workflow Diagram



Sequence Diagram



ER Diagram



7. Design Decisions & Rationale

Layered backend

Controllers, services, repositories, entities, and DTOs are separated to reduce coupling. This structure makes it easier to test business rules independently from HTTP handling and database access.

REST-based integration

The frontend communicates through a small API wrapper. The wrapper standardizes JSON headers, URL construction, handling of HTTP errors, and support for 204 No Content responses.

Relational storage

MySQL is appropriate for this domain because products, suppliers, warehouses, and movement records have clear relationships and require consistency. Foreign keys and unique SKU constraints help protect data integrity.

Transaction validation

Inward operations check warehouse remaining capacity. Outward operations check available product stock. These rules are placed in the service layer so they are not dependent on a specific frontend screen.

Dynamic alerts

The alert service calculates conditions when alerts are requested. The documented thresholds include zero stock, stock at or below the minimum level, warehouse utilization above 85%, and pending purchase orders.

8. Implementation Details

1. Frontend implementation

- React functional components with TypeScript types for Product, Warehouse, Supplier, StockInward, StockOutward, PurchaseOrder, DashboardSummary, AlertItem, and User.
- App-level state loads dashboard, products, warehouses, suppliers, inward records, outward records, purchase orders, and alerts in parallel using Promise.all.
- CRUD handlers call the API and refresh the shared data state after successful operations.
- Toast notifications communicate successful operations and API errors.
- Vite is used as the frontend development and build tool.

2. Backend implementation

- Spring Boot 3.2.3 with Java 17.
- Spring Web for REST controllers and Spring Data JPA / Hibernate for persistence.
- Validation and exception classes for invalid operations.
- ReportService exposes inventory, warehouse, inward, outward, supplier, and low-stock reporting functions.
- AlertService evaluates low stock, out-of-stock, warehouse over-utilization, and pending purchase-order conditions.
- GlobalExceptionHandler maps domain exceptions to structured error responses

API Endpoints Reference

Group	Representative Endpoints
Dashboard	<code>GET /api/dashboard/summary</code>
Products	<code>GET/POST /api/products</code> <code>PUT/DELETE /api/products/{id}</code> <code>PATCH /api/products/{id}/stock</code>
Warehouses	<code>GET/POST /api/warehouses</code> <code>PUT/DELETE /api/warehouses/{id}</code>
Suppliers	<code>GET/POST /api/suppliers</code> <code>PUT/DELETE /api/suppliers/{id}</code>
Inward	<code>GET /api/inward</code>

	<code>POST /api/inward</code>
Outward	<code>GET /api/outward</code> <code>POST /api/outward</code>
Purchase orders	<code>GET/POST /api/purchase-orders</code> <code>PATCH /api/purchase-orders/{id}/status</code>
Reports	<code>GET /api/reports/inventory, /warehouses, /inward, /outward, /suppliers, /low-stock</code>
Authentication	<code>POST /api/auth/login</code> <code>GET /api/auth/me</code>

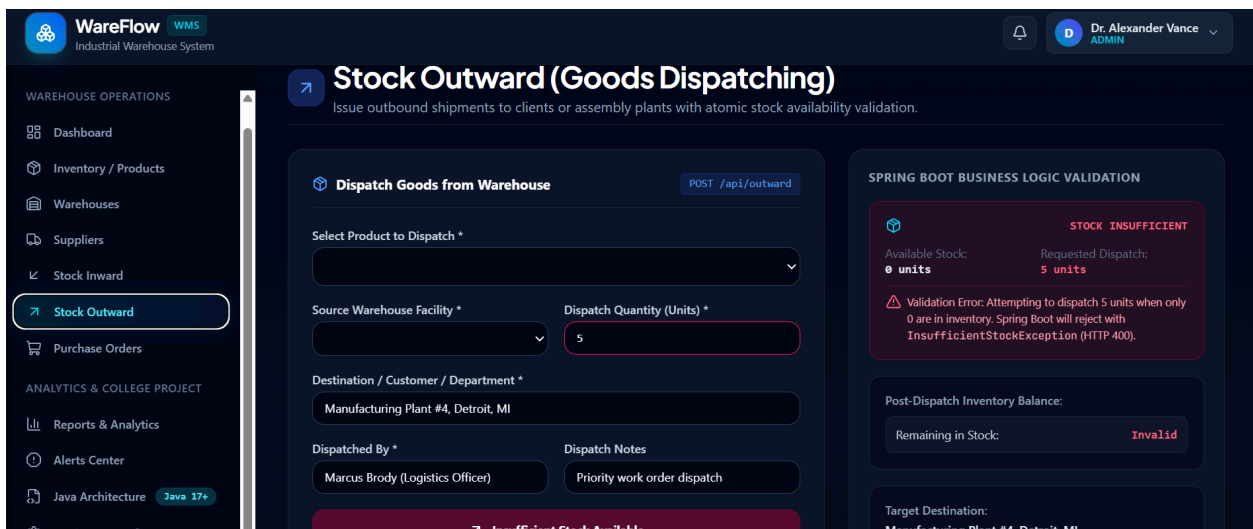
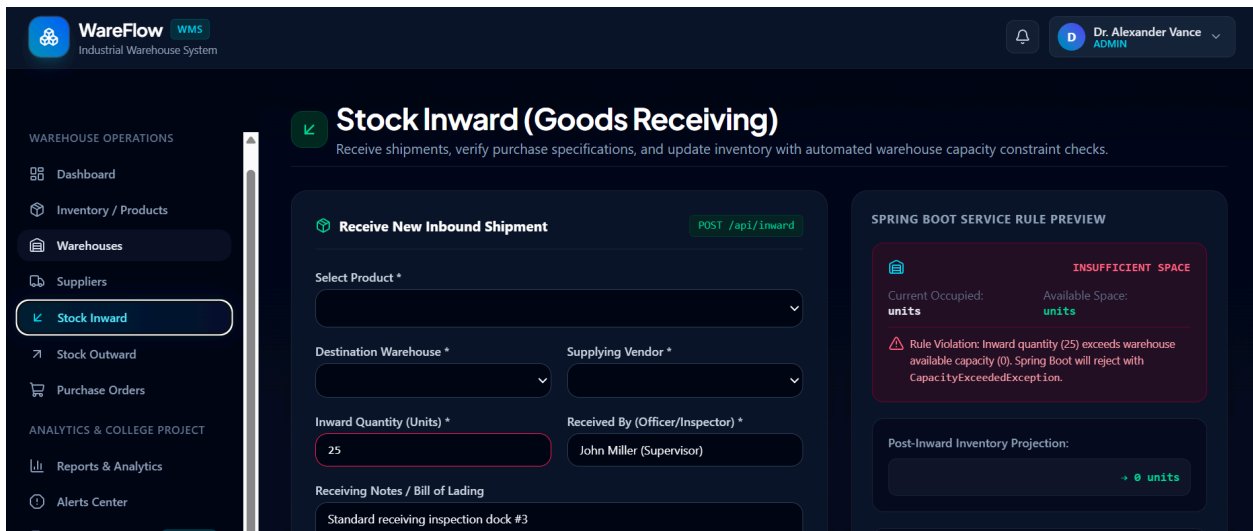
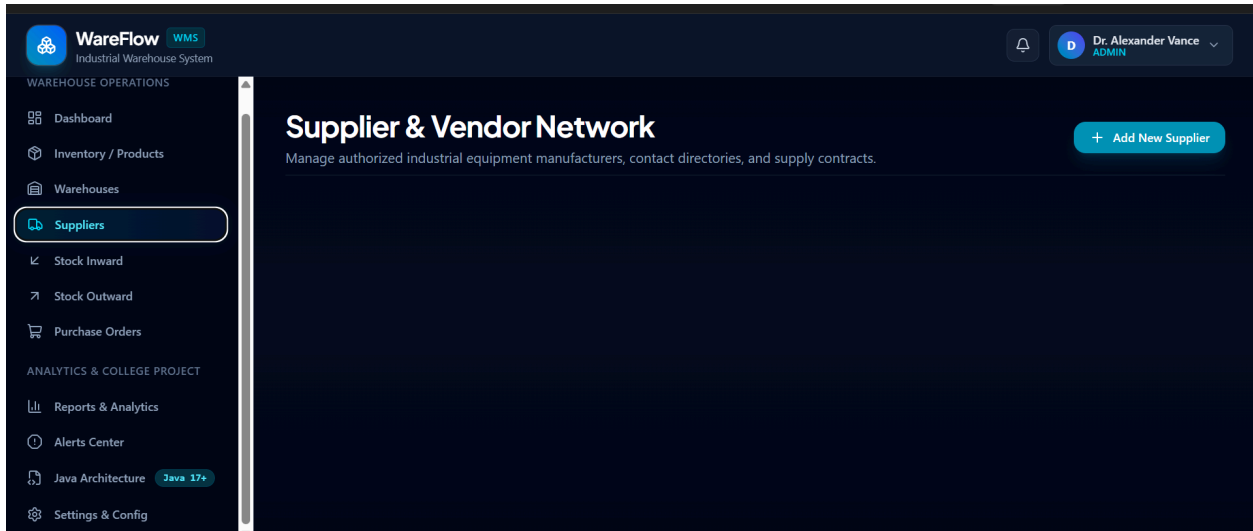
4. Database design

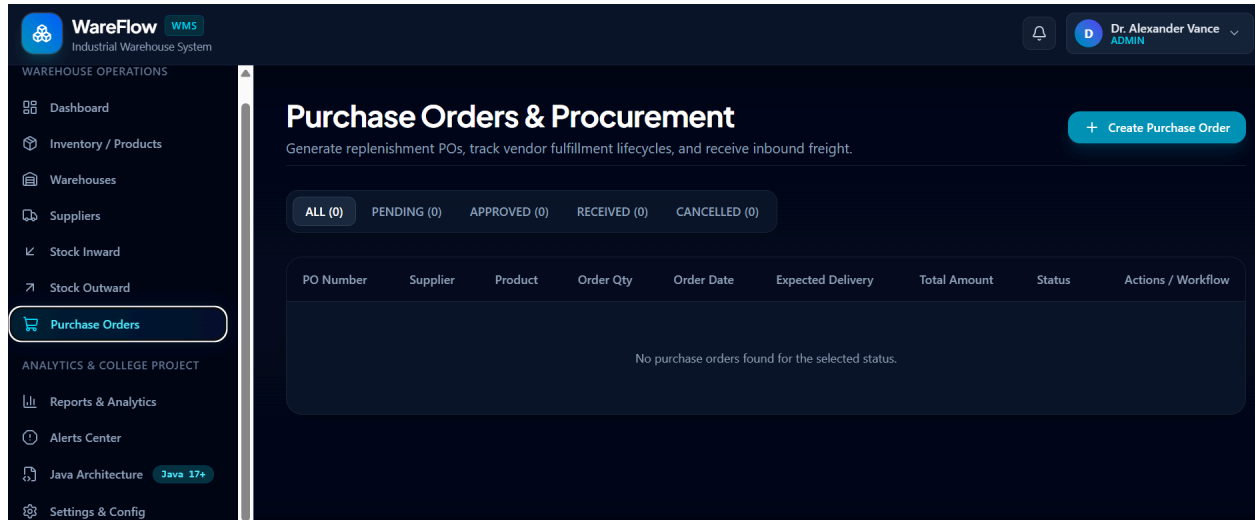
The schema defines tables for warehouses, suppliers, products, stock_inward, stock_outward, purchase_orders, and users. Product records contain SKU, category, quantity, minimum stock level, unit price, supplier reference, and warehouse reference. Movement tables preserve quantity, date, responsible user, destination or notes, and status.

9. Screenshots / Results

The screenshot shows the 'Product & Inventory Management' page in the WareFlow WMS. The left sidebar contains a navigation menu with sections for 'WAREHOUSE OPERATIONS' (Dashboard, Inventory / Products, Warehouses, Suppliers, Stock Inward, Stock Outward, Purchase Orders) and 'ANALYTICS & COLLEGE PROJECT' (Reports & Analytics, Alerts Center, Java Architecture with a 'Java 17+' badge, and Settings & Config). The main content area has a header with the WareFlow logo, 'WMS Industrial Warehouse System', a notification bell, and a user profile for 'Dr. Alexander Vance ADMIN'. Below the header is a sub-header 'Product & Inventory Management' with a description: 'Browse, search, edit SKUs, track minimum safety stock, and adjust real-time warehouse inventory.' and an '+ Add New Product' button. A search bar with the placeholder 'Search by Product Name, SKU, or Category...' is followed by a category dropdown set to 'All Categories' and filters for 'All (0)', 'Low', and 'OOS'. A table with columns 'SKU / Product', 'Category', 'Warehouse / Location', 'Current Stock', 'Safety Level', 'Unit Price', and 'Actions' is shown, but it contains no data, with a message stating 'No products found matching your search and filter criteria.'

The screenshot shows the 'Warehouse Facilities & Capacity' page in the WareFlow WMS. The left sidebar is identical to the previous screenshot. The main content area has a sub-header 'Warehouse Facilities & Capacity' with a description: 'Track total storage volume, active capacity utilization rates, and assigned facility managers.' and an '+ Add Warehouse Facility' button. The rest of the page is currently blank.





10. Testing Approach

The README states that the project does not currently ship with an automated test suite. A combined static, unit-level, integration, and manual approach is recommended.

Testing status

This report provides a test plan and expected results based on source inspection. Runtime execution results should be filled in after the application is configured with a working MySQL database and the backend and frontend are started.

11. Challenges Faced

- Coordinating frontend state with multiple backend resource collections requires a consistent refresh strategy after create, update, delete, and transaction operations.
- Inventory operations must update more than one logical value: product quantity, warehouse occupied space, available space, and movement history.
- Stock and capacity validation must be applied on the backend rather than relying only on form-level frontend validation.
- Dynamic alert generation requires threshold rules to be evaluated consistently across product and warehouse records.
- The current repository contains demo-oriented authentication behavior and permissive request authorization; production deployment would require stronger identity verification, token validation, and role-based endpoint restrictions.
- Database credentials are configured in application properties and should be externalized through environment variables or a secure configuration mechanism.

12. Learnings & Key Takeaways

- A layered architecture improves separation of concerns and makes a full-stack project easier to extend.
- DTOs are useful for shaping API contracts and reducing direct coupling between database entities and UI payloads.
- Inventory is a stateful domain: receiving and dispatching goods must be treated as validated business transactions.
- Database constraints such as unique SKU and foreign keys complement application-level validation.
- Centralized error handling produces more predictable frontend behavior.
- Dashboards and alert services turn raw operational records into information that supports warehouse decisions.
- A complete project report should distinguish implemented features, documented intentions, and features verified through runtime testing.

14. References

The report is based on the supplied project archive and its included source files.

Reference	Use in report
README.md	Project overview, features, technology stack, installation instructions, and manual testing guide.
backend/pom.xml	Java version and Spring Boot dependencies.
backend/src/main/java/com/wareflow/controller/*	REST endpoint structure.
backend/src/main/java/com/wareflow/service/*	Business logic, inventory processing, alerts, and reporting.
backend/src/main/java/com/wareflow/entity/*	Domain entities and relationships.
backend/src/main/java/com/wareflow/repository/*	Persistence interfaces.
backend/src/main/resources/schema.sql	MySQL tables, foreign keys, and indexes.
backend/src/main/resources/data.sql	Sample warehouses, suppliers, products, movements, purchase orders, and users.
src/App.tsx and src/services/api.ts	Frontend state management and API integration.
Provided project-report instruction screenshot	Required report section order.

End of Report